A

GROUP PROJECT REPORT

ON

Sound for Silence

Submitted to:

Rajasthan Technical University, Kota

In partial fulfillment of the requirement for the award of degree of

BACHELOR OF TECHNOLOGY

In

Computer Science & Engin1eering

Under the Supervision of:

Er. Pawan Sen

HOD, CSE

Submitted by:

Abhishek Verma (22EAOCS005)

Shekhar Chaurasiya (22EAOCS047)

Sakshi Puria (22EAOCS043)

Abhishek Rathore (22EAOCS004)

ARYA INSTITUTE OF ENGINEERING, TECHNOLOGY & MANAGEMENT, JAIPUR

RAJASTHAN TECHNICAL UNIVERSITY, KOTA (RAJASTHAN)

(2025–26)

ARYA INSTITUTE OF ENGINEERING, TECHNOLOGY & MANAGEMENT, JAIPUR

DEPARTMENT OF Computer Science & Engin1eering

Certificate of Completion

This is to certify that the project titled Sound for Silence has been successfully completed by the following students of Bachelor of Technology, 8th Semester, at Arya Institute of Engineering, Technology & Management, Jaipur, in partial fulfillment of the academic requirements for the 8th semester of the B.Tech program. The project work presented in this report is a bonafide and satisfactory account of the work carried out under our supervision.

Project In-charge:

Submitted by:

Abhishek Verma (22EAOCS005)

Shekhar Chaurasiya (22EAOCS047)

Sakshi Puria (22EAOCS043)

Abhishek Rathore (22EAOCS004)

DECLARATION BY THE CANDIDATES

We, **Abhishek Verma, Shekhar Chaurasiya, Sakshi Puria,** and **Abhishek Rathore**, hereby declare that the project work titled Sound for Silence is an original and authentic piece of work carried out by us at Arya Institute of Engineering, Technology & Management, Jaipur in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (B.Tech) in Computer Science & Engin1eering.

We further affirm that this work has not been submitted, either fully or partially, for the award of any other degree or diploma at any institution. We have attributed all sources and acknowledged any third-party contributions as per academic integrity guidelines.

Submitted by:

Abhishek Verma (22EAOCS005)

Shekhar Chaurasiya (22EAOCS047)

Sakshi Puria (22EAOCS043)

Abhishek Rathore (22EAOCS004)

ACKNOWLEDGEMENT

It gives us immense pleasure to express my heartfelt gratitude to all those who have supported and guided me throughout the successful completion of this final year project.

I would first like to extend my sincere thanks to **Dr. Arvind Agarwal**, President, and **Dr. Puja Agarwal**, Group Director of Arya Institute of Engineering, Technology & Management, Jaipur, for providing an excellent academic environment and all the necessary facilities that foster learning and innovation.

I am deeply grateful to our respected Principal, **Dr. Himanshu Arora**, for his continuous encouragement and administrative support. I would also like to express my special thanks to **Er. Pawan Sen**, Head of the Department, **Computer Science and Engineering**, for his visionary leadership, valuable suggestions, and motivation throughout the course of this project.

I am also thankful to my Project In-charge, **Er. Pawan Sen**, whose timely feedback, guidance, and coordination played a vital role in ensuring the quality and completion of this project work.

I extend my gratitude to my project teammates for their collaborative efforts and coordination throughout the development cycle. The division of roles — Android Development, Web Dashboard Development, Backend & Firebase Engineering, and UI/UX Design & Testing — allowed each member to contribute meaningfully and ensured smooth parallel progress across the system.

Last but not least, I sincerely thank all the faculty members, lab assistants, and my peers who contributed directly or indirectly with their support and inspiration during the entire project duration.

This project has been a significant learning experience and a stepping stone in my professional development.

Submitted by:

Abhishek Verma (22EAOCS005)

Shekhar Chaurasiya (22EAOCS047)

Sakshi Puria (22EAOCS043)

Abhishek Rathore (22EAOCS004)

LIST OF TABLES

Table 1 Backend Architecture Components and Technologies 8
Table 2 Firebase Services Free Tier Limits and Project Usage 9
Table 3 Users Collection Schema Fields 21
Table 4 Abbreviations and Acronyms 61

LIST OF IMAGES

Figure 1 High-Level System Architecture Overview 26
Figure 2 Android App: Video Playback (Left) and Parent Performance Dashboard (Right) 29
Figure 3 Web Dashboard: Patient List (Left), Patient History/Progress Chart (Middle), Assessment Score Entry (Right) 30

TABLE OF CONTENTS

1. Introduction 1
    1.1. 1.1 Abstract of Project 1
    1.2. 1.2 Problem Statement 3
    1.3. 1.3 Project Objectives 3
    1.4. 1.4 Project Scope 4
    1.5. 1.5 Organization of the Report 4
2. Literature Review 6
    2.1. 2.1 Related Work and Existing Technologies 6
    2.2. 2.2 Existing Platforms and Tools Analyzed 8
    2.3. 2.3 Gaps in Existing Solutions 9
3. Feasibility Study 11
    3.1. 3.1 Technical Feasibility 11
    3.2. 3.2 Operational Feasibility 12
    3.3. 3.3 Economic Feasibility 13
4. Methodology / Planning of Work 15
    4.1. 4.1 Chosen Methodology: Agile Software Development Lifecycle (SDLC) 15
    4.2. 4.2 Project Phases and Planning 15
    4.3. 4.3 Testing Strategy 17
    4.4. 4.4 Tools and Technologies 17
5. Facilities Required for Proposed Work 19
    5.1. 5.1 Hardware Requirements 19
    5.2. 5.2 Software Requirements 20
    5.3. 5.3 External Services and APIs 20
6. Design 22
    6.1. 6.1 System Architecture 22
    6.2. 6.2 Data Model Design (Firestore) 27
    6.3. 6.3 Data Flow Diagrams (DFD) 28
    6.4. 6.4 User Interface (UI) Design and Screenshots 29
    6.5. 6.5 Security Design Considerations 30
7. Implementation / Technological Environment 32
    7.1. 7.1 Development Environment Setup 32
    7.2. 7.2 Backend Implementation with Firebase 33

7.3. 7.3 Android Application Implementation Details 34
7.4. 7.4 Web Dashboard Implementation Details 35
7.5. 7.5 Integration and Synchronization 36
8. Testing and Results 37
8.1. 8.1 Unit Testing 37
8.2. 8.2 Integration Testing 37
8.3. 8.3 End-to-End (E2E) Testing 38
8.4. 8.4 Usability Testing 38
8.5. 8.5 Results Summary 39
9. Limitations 40
9.1. 9.1 Technical Limitations 40
9.2. 9.2 Operational Limitations 40
9.3. 9.3 Clinical Limitations 41
10. Conclusion and Future Scope 42
10.1. 10.1 Conclusion 42
10.2. 10.2 Future Scope 42
11. Bibliography 45
A.1 Android - Key Composables and ViewModel Snippets (Kotlin/Compose) . 46
A.2 Web Dashboard - JavaScript Snippets for Firebase, Chart.js, and UI Handling 54
A.3 Typst Code Snippet: Custom Callout Box 59

Introduction

1.1 Abstract of Project

The AI Adaptive Learning Platform is a sophisticated, dual-interface digital healthcare solution designed to revolutionize therapy for children undergoing cognitive, auditory, and speech rehabilitation. This platform specifically targets children who have received cochlear implants and require consistent, monitored therapeutic intervention over extended periods. It aims to provide a structured, engaging, and data-driven environment that enhances learning outcomes for these young individuals, while offering transparent progress visibility to parents and robust clinical tools to therapists.

The system is comprised of two distinct yet interconnected interfaces: an Android mobile application and a web-based dashboard. The Android application, developed using Kotlin and Jetpack Compose, serves as a comprehensive Patient and Parent Portal. It delivers therapy content through a series of gamified learning stages, unlocked sequentially via video modules. Mastery of each video is reinforced by a short, comprehension-testing quiz. Parents can log in to monitor their child's journey, tracking metrics such as daily streaks, stage completion percentages, and overall performance. This interface is designed with a game-like aesthetic, featuring large buttons and bright visuals to maximize engagement with young users.

Complementing the mobile application, the web dashboard is built with standard web technologies (HTML, CSS, JavaScript) and serves as a Therapist Portal. Therapists can log in to manage their patient roster, access patient records, schedule and record crucial clinical scores like the CAP score (Categories of Auditory Performance, 0–7) and the SIR score (Speech Intelligibility Rating, 0–5) for each session. The dashboard integrates Chart.js to generate longitudinal progress charts, allowing therapists to visualize patient improvement over 3-month, 6-month, and 12-month periods. This data-driven approach empowers therapists with insights for informed clinical decision-making.

The entire backend infrastructure is unified by Firebase, a powerful managed cloud platform. Firebase Authentication provides secure, role-based login for both parents and therapists, ensuring data privacy and system integrity. Firebase Firestore is utilized as the NoSQL database to store all application data, including user profiles, patient records,

stage content, video metadata, quiz results, and clinical assessment data. Firebase Hosting efficiently serves the web dashboard, eliminating the need for dedicated server management.

This report details the collective efforts of our group, comprising four members, each contributing expertise in specialized areas: Frontend Development, Backend & AI Integration Engineering, and Database & System Design. The project's core objective is to bridge the gap left by traditional, often manual, therapy tracking methods, which are prone to errors and lack longitudinal visibility. By offering a structured learning environment, transparent progress tracking, and data-driven clinical tools, Sound for Silence aims to significantly improve the efficacy and accessibility of therapy for children with cochlear implants.

1.1.1 Project Title

AI Adaptive Learning Platform

1.1.2 Project Rationale and Significance

Children with cochlear implants require consistent, guided practice over months and years to achieve optimal therapeutic outcomes. Currently, therapy progress is often tracked manually using paper records or disparate digital tools, a method that is notoriously difficult to monitor longitudinally and highly prone to error and data loss. This fragmented approach often leads to a lack of transparency for parents regarding their child's progress and can overwhelm therapists with administrative burdens, detracting from their primary focus on patient care.

The AI Adaptive Learning Platform directly addresses these critical shortcomings. It provides a structured, engaging, and gamified learning environment for the child, intrinsically motivating them through sequential video lessons and interactive quizzes. Concurrently, it offers transparent progress visibility to parents, allowing them to actively participate in and support their child's therapeutic journey. For therapists, the platform delivers data-driven clinical tools, simplifying patient management, enabling precise recording of key performance indicators (CAP and SIR scores), and providing powerful visualization of long-term progress through interactive charts.

By centralizing data and streamlining workflows, the platform not only enhances the therapeutic process but also significantly reduces administrative overhead for healthcare providers. The use of modern web and mobile technologies, coupled with AI for content generation and adaptive learning, ensures the platform is scalable, accessible, and capable of evolving with the needs of its users. This project represents a critical step towards modernizing pediatric audiology and speech therapy through technology, making rehabilitation more effective, accessible, and data-informed for all stakeholders.

1.2 Problem Statement

The current methods for tracking the rehabilitation progress of children with cochlear implants are largely manual, fragmented, and lack longitudinal data integration. This results in:

1. Inconsistent Practice: Children may not receive consistent, guided practice due to the difficulty of managing and motivating them with traditional methods.
2. Lack of Visibility: Parents often have limited insight into their child's daily engagement, progress, and specific areas of difficulty.
3. Administrative Burden: Therapists spend significant time on manual record-keeping, score entry, and progress assessment, diverting time from direct patient interaction and clinical analysis.
4. Data Silos: Progress data, when collected, is often stored in separate paper records or disconnected digital files, preventing comprehensive longitudinal analysis.
5. Limited Engagement: Traditional therapy materials may not be sufficiently engaging for young children, leading to decreased adherence and suboptimal outcomes.

1.3 Project Objectives

The primary goal of the AI Adaptive Learning Platform is to create a unified, intelligent, and engaging ecosystem for pediatric auditory and speech therapy. This will be achieved through the following specific objectives:

Develop a Gamified Android Application: To create an interactive Android application for children that delivers sequential, gamified therapy content through video modules and comprehension-based quizzes. This app will automatically unlock subsequent stages upon successful completion of prerequisites, ensuring a structured learning path.

Build a Comprehensive Web Dashboard: To develop a web-based portal for therapists that enables efficient management of patient records, facilitates the recording of clinical scores (e.g., CAP and SIR scores), and provides interactive charts for visualizing long-term patient progress over various timeframes (3, 6, 12 months). Implement a Unified Firebase Backend: To establish a secure, role-based backend using Firebase Authentication and Firestore. This backend will manage user authentication for parents and therapists and ensure real-time data synchronization between the Android app and the web dashboard, thereby eliminating the need for a dedicated server infrastructure. Design a Transparent Parent Portal: To provide parents with a clear, easily accessible portal that offers visibility into their child's daily therapy activities, quiz performance, and overall rehabilitation progress, fostering parental involvement and support. Ensure Data Integrity and Accessibility: To store all application data securely and reliably in Firebase Firestore, making it accessible in real-time across both interfaces and enabling data-driven clinical decision-making.

1.4 Project Scope

The scope of this project encompasses the development of two client interfaces (Android application and Web dashboard) and a unified backend managed by Firebase. Android Application (Patient/Parent Portal): Focuses on delivering gamified therapy content, sequential learning, basic quiz assessment, and parent progress tracking. Web Dashboard (Therapist Portal): Focuses on patient management, clinical score recording, and visualization of longitudinal progress data. Backend: Provides secure authentication, real-time data synchronization between clients, and data storage using Firebase services.

The project will not include features such as: advanced AI-driven content generation (the content is pre-defined or curated), complex diagnostic tools beyond CAP/SIR scoring, integration with external Electronic Health Record (EHR) systems, or advanced analytics dashboards for therapists beyond progress charts. The focus is on providing a robust, functional, and user-friendly platform for the specified scope.

1.5 Organization of the Report

This report is structured to provide a comprehensive overview of the AI Adaptive Learning Platform project. Chapter 1: Introduction: Provides an abstract, rationale,

problem statement, objectives, and scope of the project. Chapter 2: Literature Review: Explores existing technologies, related systems, and research relevant to the project's domain. Chapter 3: Feasibility Study: Analyzes the technical, operational, and economic viability of the proposed system. Chapter 4: Methodology / Planning of Work: Details the development lifecycle, planning, and chosen methodologies. Chapter 5: Facilities Required: Lists the hardware, software, and services necessary for project development. Chapter 6: Design: Outlines the system architecture, data models, and user interface designs, incorporating dashboard screenshots. Chapter 7: Implementation: Describes the technologies used and key aspects of the development process. Chapter 8: Testing and Results: Details the testing strategies employed and the outcomes achieved. Chapter 9: Limitations: Discusses the current constraints and areas for future improvement. Chapter 10: Conclusion and Future Scope: Summarizes the project's achievements and outlines potential future enhancements. Bibliography: Lists all referenced sources. Appendices: Includes supplementary materials such as code snippets or specific diagrams if applicable.

Literature Review

This section critically examines existing research, technologies, and applications relevant to the AI Adaptive Learning Platform, providing context for our project's design choices and identifying gaps that our solution aims to fill.

2.1 Related Work and Existing Technologies

The field of digital healthcare and educational technology for children, particularly those with specific therapeutic needs like cochlear implant recipients, has seen significant advancements. This review focuses on key areas: mobile health (mHealth) applications, gamified learning, adaptive learning systems, and backend technologies suitable for real-time data synchronization.

2.1.1 Mobile Health (mHealth) Applications in Pediatric Therapy

Mobile applications have emerged as powerful tools for delivering and monitoring healthcare interventions, especially for pediatric populations. Studies have shown that mHealth interventions can improve adherence to treatment protocols and provide valuable data for clinicians. For instance, applications designed for children with chronic conditions often incorporate elements of gamification to increase engagement and motivation. Research in pediatric audiology highlights the potential of mobile apps to deliver structured auditory training and track progress, offering a more accessible and consistent alternative to traditional clinic-based sessions. However, many existing mHealth solutions for therapy either lack robust parent-involvement features or fail to provide therapists with comprehensive, longitudinal data for clinical assessment. This gap underscores the need for integrated platforms that serve all stakeholders effectively.

2.1.2 Gamified Learning and Engagement Strategies

Gamification, the application of game-design elements and game principles in non-game contexts, has proven highly effective in educational and therapeutic settings, particularly for children. Elements such as points, badges, leaderboards, progressive unlocking of content, and immediate feedback loops can significantly boost user motivation, engagement, and retention. In the context of auditory and speech therapy,

gamified elements can transform repetitive practice exercises into enjoyable activities, encouraging consistent participation. The effectiveness of gamification is often tied to personalized feedback and clear progress visualization, allowing users to see the direct impact of their efforts.

2.1.3 Adaptive Learning Systems and Content Personalization

Adaptive learning systems adjust the presentation of material and the sequence of learning activities based on an individual learner's performance and needs. While our project focuses on a pre-defined, sequential learning path rather than dynamic content adaptation, the principles of adaptive systems—monitoring progress, providing feedback, and adjusting difficulty—are relevant. For children with cochlear implants, adaptive features could include varying the complexity of auditory tasks or speech intelligibility exercises based on their performance metrics. Such systems require sophisticated backend logic and robust data analysis capabilities.

2.1.4 Backend Technologies for Real-time Data Synchronization

The need for real-time data synchronization between mobile and web applications is critical in healthcare and educational platforms. Technologies that facilitate this include: Firebase: A comprehensive backend-as-a-service (BaaS) platform by Google, offering real-time NoSQL databases (Firestore), authentication, cloud functions, and hosting. Firebase is well-suited for projects requiring rapid development, cross-platform synchronization, and scalability without the overhead of managing dedicated servers. Its Firestore database provides real-time listeners, ensuring that updates made on one client are instantly reflected on others. Backend-as-a-Service (BaaS) Platforms: Alternatives like AWS Amplify, Parse Server, or Supabase offer similar functionalities, providing managed databases, authentication, and APIs. The choice of Firebase was driven by its user-friendly interface, robust free tier for development, and seamless integration with Android and web development stacks.

Component	Technology	Purpose
Authentication	Firebase Authentication	Email/password login, JWT tokens, custom role claims
Database	Cloud Firestore (NoSQL)	All persistent application data storage and real-time sync
Security	Firestore Security Rules	Role-based access control at database layer
Hosting	Firebase Hosting	Static deployment of web dashboard with CDN and HTTPS
Admin Operations	Firebase Admin SDK (Node.js)	Setting custom claims, seeding initial data
Client SDK — Android	Firebase Android SDK (BOM 32.x)	Auth and Firestore access from Kotlin app
Client SDK — Web	Firebase JS SDK v9 (Modular)	Auth and Firestore access from JS dashboard
Project Config	google-services.json	Android Firebase project configuration

Table 1: Backend Architecture Components and Technologies

Custom Backend (Node.js/Express with WebSockets): While more complex, a custom backend using Node.js with libraries like Socket.IO can provide greater control and flexibility. However, for a student project with limited resources and a focus on rapid development, BaaS platforms like Firebase offer a significant advantage.

2.2 Existing Platforms and Tools Analyzed

During the project's initial research phase, several existing tools and platforms were evaluated to understand their features, limitations, and potential applicability.

Service	Free Tier Limit	Project Usage
Firebase Authentication	Unlimited (email/password)	< 100 users for development
Cloud Firestore	50,000 reads/day, 20,000 writes/day, 1 GB storage	Sufficient for development and demo
Firebase Hosting	10 GB storage, 360 MB/day transfer	Well within limit for dashboard
Firebase Admin SDK	Free (runs client-side or via Cloud Functions)	Used for custom claims setup

Table 2: Firebase Services Free Tier Limits and Project Usage

The analysis of existing solutions revealed several key insights: Integration is Key: Many applications focus on either the child's engagement or the therapist's tools, but few successfully integrate both seamlessly with parent involvement. The need for a unified platform that bridges these user roles is evident. Data-Driven Therapy: Therapists benefit immensely from objective, longitudinal data. The lack of such features in many existing apps limits their clinical utility. Tools like Chart.js are valuable for visualization, but require robust data aggregation. Firebase Advantage: Firebase's real-time capabilities and integrated services (Authentication, Firestore, Hosting) offer a compelling solution for projects requiring cross-platform synchronization and a managed backend, significantly reducing development complexity and cost. Gamification for Engagement: The use of gamified elements is crucial for maintaining the attention and motivation of young children in therapy.

Our project aims to address these identified gaps by providing a holistic solution that integrates child engagement, parent visibility, and therapist clinical tools within a single, cohesive platform, powered by a robust and scalable backend.

2.3 Gaps in Existing Solutions

Based on the literature review and analysis of existing platforms, several significant gaps were identified that our project aims to address:

Lack of Integrated Stakeholder Experience: Many existing solutions cater to one primary user group (e.g., therapists, or children) but fail to provide a cohesive experience for all stakeholders – the child, the parent, and the therapist. Often, progress

data is not transparently shared with parents or therapists in a timely manner. Absence of Standardized Clinical Scoring: While some apps track progress, few incorporate standardized clinical assessment scores like CAP (Categories of Auditory Performance) and SIR (Speech Intelligibility Rating) directly into their tracking mechanisms. This limits the clinical validity and utility of the data collected. Manual Data Entry and Error-Proneness: Traditional paper-based record-keeping or disjointed digital logs are time-consuming and susceptible to errors, making longitudinal analysis difficult and potentially impacting clinical accuracy. Limited Longitudinal Visualization: Therapists often struggle to visualize patient progress over extended periods (e.g., 3-12 months) in a clear, actionable format. Existing tools may offer basic charts but lack the specific clinical focus needed for therapy. Insufficient Parental Engagement Tools: While some apps allow parents to view basic progress, few provide deep insights or tools for parents to actively participate in or support their child's therapy sessions from home. Backend Complexity and Server Management: Many advanced features require complex back-end infrastructure, which can be prohibitive for student projects in terms of development time, cost, and maintenance.

Our project aims to bridge these gaps by providing an integrated platform that synchronizes data in real-time between a gamified child-focused Android app and a data-rich therapist-focused web dashboard. It incorporates standardized clinical scoring, visualizes long-term progress, and enhances parental engagement, all while leveraging a managed backend service (Firebase) to simplify development and deployment.

Feasibility Study

This section evaluates the practicality and viability of the proposed AI Adaptive Learning Platform, focusing on technical, operational, and economic aspects. The project is designed to be achievable within the scope of a student project while also laying a foundation for potential real-world application.

3.1 Technical Feasibility

The AI Adaptive Learning Platform is technically feasible due to the utilization of widely adopted, well-documented, and robust technologies.

Android Application Development: The Android app is being developed using Kotlin and Jetpack Compose. Kotlin is the modern, preferred language for Android development, known for its conciseness and safety features. Jetpack Compose, Android's modern UI toolkit, enables declarative UI design, making it easier to build responsive and performant interfaces. Both technologies are officially supported by Google, with extensive documentation, community support, and readily available libraries. The use of ExoPlayer (via Media3) for video playback is standard and highly reliable. The Firebase Android SDK provides seamless integration for authentication, database operations, and offline data caching.

Web Dashboard Development: The web dashboard will be built using standard web technologies: HTML for structure, CSS for styling, and JavaScript for interactivity. This approach ensures maximum compatibility across modern web browsers. The Firebase JS SDK (v9 modular) will be employed for client-side interaction with Firebase services. Chart.js will be used for rendering interactive progress charts, a well-established and versatile JavaScript charting library. Firebase Hosting provides a simple and scalable solution for deploying the static web assets.

Backend Infrastructure (Firebase): The entire backend is managed by Firebase. This includes: Firebase Authentication: Offers secure, out-of-the-box solutions for email/ password login, role-based access control (via custom claims), and secure user management, significantly reducing backend development effort and security concerns. Firebase Firestore: A NoSQL, real-time document database that is highly scalable and provides offline support for the Android app. Its real-time listeners ensure instant data

synchronization between the Android app and the web dashboard, eliminating the need for custom server-side synchronization logic. Firebase Hosting: Provides a reliable and performant global CDN for serving the web dashboard.

The combination of these technologies is proven, widely used in industry, and well within the capabilities of a student team with foundational knowledge in mobile, web, and cloud services. The use of Firebase drastically simplifies backend development and infrastructure management, making the project technically achievable. The learning curve for Firebase integration is manageable, and extensive documentation and community resources are available.

3.2 Operational Feasibility

The AI Adaptive Learning Platform is designed for high operational feasibility, focusing on simplicity, intuitiveness, and ease of use for all intended user groups.

Child-Centric Design (Android App): The Android application is tailored for young users with cochlear implants who may have cognitive, auditory, or speech-related challenges. It employs a gamified interface with large, easily tappable buttons, bright and appealing visuals, and sequential learning paths that guide the child through therapy modules. The content delivery is structured with unlocked stages, ensuring a manageable and non-overwhelming experience. Short, comprehension-focused quizzes reinforce learning after each video segment. This design prioritizes engagement and reduces the cognitive load on the child.

Parental Engagement Portal: The parent portal within the Android app provides clear visibility into their child's daily therapy activities. Parents can easily track metrics such as daily streaks (encouraging consistent practice), stage completion percentages (showing overall progress), and performance on quizzes. This transparency aims to foster parental involvement, allowing them to support their child's learning journey more effectively. The interface is designed to be intuitive, requiring no technical expertise from the parent.

Therapist Dashboard Usability: The web dashboard is engineered for therapist efficiency. Therapists can quickly access their patient list, view individual patient records, and manage upcoming assessment schedules. The interface for recording clinical scores

(CAP and SIR) is straightforward, minimizing data entry time. Crucially, progress visualization using Chart.js provides an at-a-glance overview of a patient's improvement over time, facilitating data-driven clinical decisions without complex data manipulation. The web interface is responsive and accessible from any modern browser.

Data Synchronization and Reduced Manual Work: The real-time data synchronization provided by Firebase Firestore eliminates the need for manual data entry or transfer between different systems (e.g., paper records to digital logs). This automation significantly reduces the risk of errors, saves time for both parents and therapists, and ensures that all stakeholders are working with the most up-to-date information.

Accessibility: The platform is designed with accessibility in mind, particularly for children with therapeutic needs. While full WCAG compliance is a complex undertaking, the focus on clear UI, large touch targets, and sequential content aids users with potential cognitive or motor challenges.

Overall, the operational feasibility is high, as the system is designed to be intuitive for children, informative for parents, efficient for therapists, and requires minimal technical overhead for all users.

3.3 Economic Feasibility

The AI Adaptive Learning Platform project is highly economically feasible, primarily due to its strategic reliance on open-source technologies and a generous free tier provided by its chosen backend platform, Firebase.

Open-Source Software: Android Development: Kotlin and Jetpack Compose are free and open-source, officially supported by Google. Android Studio, the primary Integrated Development Environment (IDE), is also free. Web Development: HTML, CSS, and JavaScript are fundamental web technologies and are free to use. Chart.js, the charting library, is open-source. Visual Studio Code, a popular IDE, is also free. Backend (Firebase): Firebase offers a substantial free tier (Spark Plan) that covers most of the project's needs during development and for small-scale deployments. This includes: Firebase Authentication: Free for unlimited users. Firebase Firestore: Generous free quota for reads, writes, and storage, sufficient for testing and initial user bases. Firebase Hosting: Free hosting for web applications with global CDN. The only potential cost

for the Android app itself is the one-time Google Play Store developer account fee (approximately USD 25), which is minimal for app distribution.

Minimal Infrastructure Costs: By utilizing Firebase, the project bypasses the need for procuring, configuring, and maintaining dedicated servers. This eliminates significant infrastructure costs associated with traditional backend development (e.g., server hardware, cloud computing instances, database administration).

Scalability and Cost Management: As the platform scales beyond the free tier limits, Firebase offers pay-as-you-go pricing. Costs are directly tied to usage (e.g., data storage, read/write operations, bandwidth), allowing for controlled scaling and budget management. For a student project, the free tier is more than adequate.

In conclusion, the economic feasibility is exceptionally high. The project can be developed and deployed with minimal to zero financial investment, making it an ideal candidate for academic projects and a cost-effective solution for healthcare providers, especially those in resource-limited settings.

Methodology / Planning of Work

The development of the AI Adaptive Learning Platform follows a structured yet flexible approach, leveraging modern software development methodologies to ensure efficient progress, high quality, and timely delivery. Our chosen methodology is rooted in Agile principles, adapted for a group project context with distinct development streams for the Android app, web dashboard, and backend services.

4.1 Chosen Methodology: Agile Software Development Lifecycle (SDLC)

We have adopted an Agile Software Development Lifecycle (SDLC) approach, specifically incorporating elements of Scrum. Agile methodologies are chosen for their iterative nature, emphasis on collaboration, rapid prototyping, and responsiveness to change, which are crucial for a project involving multiple development facets and user groups.

Iterative Development: The project is broken down into several iterations or “sprints,” typically lasting two weeks. Each sprint focuses on delivering a set of working features for either the Android app, the web dashboard, or the backend integration. Collaboration and Communication: Regular team meetings (daily stand-ups, sprint planning, sprint reviews, and retrospectives) are conducted to ensure seamless communication and coordination among team members responsible for different components (Android, Web, Backend). User Feedback Integration: Although direct user feedback might be limited during development, the project plan incorporates stages for review by faculty guides and simulated user testing to incorporate feedback early and often. Adaptability: The Agile approach allows for flexibility in responding to any unforeseen challenges or changes in requirements, ensuring the project remains aligned with its objectives.

4.2 Project Phases and Planning

The project is systematically planned and executed through distinct phases:

Phase 1: Requirement Analysis and Definition (Completed)

Activities: Detailed study of project scope, objectives, and user needs (child, parent, therapist). Definition of functional and non-functional requirements. Identification of key technologies and platform choices. Deliverables: Project proposal document, finalized requirement specifications, initial use case diagrams. Duration: 1-2 Weeks.

Phase 2: Design and Architecture (Completed)

Activities: High-level system architecture design (client-server model with Firebase backend). Database schema design for Firestore. User interface (UI) and user experience (UX) design for both Android app and web dashboard. Planning for Firebase integration (Authentication, Firestore, Hosting). Deliverables: System architecture diagrams, database schema documentation, UI mockups/wireframes, technical design documents. Duration: 2-3 Weeks.

Phase 3: Development and Implementation (Ongoing)

This phase is executed in iterative sprints:

Sprint 1: Backend Foundation & Authentication: Set up Firebase project, configure Authentication, and implement basic user roles. Develop initial Firestore structure for users and basic patient/therapist profiles. Implement secure login and registration flows. Deliverable: Core authentication working for both web and Android.

Sprint 2: Core Data Model & Synchronization: Implement Firestore data models for therapy stages, videos, and quiz results. Set up real-time listeners for data synchronization between clients. Develop basic CRUD operations for therapy content management. Deliverable: Real-time data sync for core content working.

Sprint 3: Android App - Content Delivery: Develop UI for displaying therapy stages and sequential videos. Integrate video playback functionality. Implement quiz interface and basic scoring logic. Develop parent portal view for progress tracking. Deliverable: Functional Android app for content delivery and basic parent view.

Sprint 4: Web Dashboard - Therapist Tools: Develop therapist login and patient list view. Implement patient detail view and score recording interface (CAP/SIR). Integrate Chart.js for progress visualization. Deliverable: Functional web dashboard for therapists.

Sprint 5: Integration, Refinement & Gamification: Ensure seamless integration and data flow between Android and Web. Refine UI/UX based on internal testing. Implement gamification elements in the Android app (streaks, completion percentages). Deliverable: Integrated platform with refined UI and gamified features.

Sprint 6: Testing, Deployment & Documentation: Conduct comprehensive end-to-end testing. Prepare for deployment to Firebase Hosting and Google Play Store. Finalize project documentation. Deliverable: Deployed application and final report.

Duration: 8-12 Weeks (broken into sprints).

4.3 Testing Strategy

Testing is an integral part of each phase to ensure quality and reliability.

Unit Testing: Individual components and functions (e.g., score calculation logic, data validation) are tested in isolation. For Android, JUnit and Mockito are used. For web, Jest or Mocha can be employed. Integration Testing: Focuses on verifying the interaction between different modules and services. This includes testing the data flow between the Android app and Firebase, the web dashboard and Firebase, and ensuring real-time synchronization works correctly. End-to-End (E2E) Testing: Simulates real user scenarios across the entire platform. For example: A therapist logs in, adds a patient, records scores, and views progress charts. A parent logs in, views their child's activity for the day, and checks completion percentages. A child completes a video and a quiz on the Android app, and the progress reflects on the parent portal and therapist dashboard. User Acceptance Testing (UAT): Although direct UAT might be limited, internal testing by team members acting as different user roles will simulate UAT. Faculty advisors will also provide feedback. Firebase Security Rule Testing: Thoroughly test security rules to ensure data access is restricted according to user roles.

4.4 Tools and Technologies

Mobile App: Kotlin, Jetpack Compose, Android Studio, Firebase Android SDK, ExoPlayer (Media3). Web Dashboard: HTML, CSS, JavaScript (ES6+), Firebase JS SDK v9, Chart.js, Visual Studio Code. Backend & Database: Firebase (Authentication,

Firestore, Hosting). Version Control: Git, GitHub. Collaboration: Discord/Slack for communication, GitHub for code repository.

Facilities Required for Proposed Work

The development of the AI Adaptive Learning Platform necessitates specific hardware, software, and access to cloud services to ensure efficient progress and the successful realization of the project goals. The requirements are standard for modern software development and are expected to be readily available within an academic or professional environment.

5.1 Hardware Requirements

Development Machines: Each team member will require a personal computer (desktop or laptop) capable of running modern development tools. Processor: Intel Core i5 or equivalent (quad-core or higher recommended). RAM: Minimum 8 GB RAM, with 16 GB strongly recommended for smoother performance, especially when running Android emulators, multiple browser tabs, and IDEs simultaneously. Storage: At least 256 GB SSD (Solid State Drive) is recommended for faster project loading, compilation, and emulator performance. 512 GB or more is preferable. Display: A standard monitor resolution (e.g., 1920x1080) is sufficient. Dual monitors can enhance productivity for developers working on both frontends and backend integration. Internet Connectivity: A stable, high-speed broadband internet connection is crucial for accessing Firebase services, downloading dependencies, and collaborating effectively.

Testing Devices: Android Device: While Android emulators in Android Studio are useful, a physical Android device is highly recommended for real-world testing of the mobile application, especially for performance profiling and accurate testing of audio/video playback and camera functionalities (if any were planned). The device should ideally be running a recent version of Android to match potential user environments. Web Browsers: Access to modern web browsers (e.g., Google Chrome, Mozilla Firefox, Microsoft Edge) on a desktop machine is required for testing the web dashboard's responsiveness and cross-browser compatibility.

Cloud Infrastructure (Managed by Firebase): The project leverages Firebase, which provides a managed cloud infrastructure. This means dedicated server hardware, complex network configurations, and database servers are not required from the team's side. The

necessary infrastructure for hosting the web dashboard, running authentication services, and managing the Firestore database is handled by Google's cloud platform.

5.2 Software Requirements

Operating Systems: Windows 10/11, macOS (latest versions recommended), or Linux (e.g., Ubuntu 20.04 LTS or later) for development machines.

Integrated Development Environments (IDEs): Android Development: Android Studio (latest stable version). This IDE includes the Android SDK, emulator, build tools, and Kotlin/Compose support. Web Development: Visual Studio Code (latest stable version) is recommended for its lightweight nature, extensive extension ecosystem (for HTML, CSS, JavaScript, Typst), and integrated terminal.

Programming Languages & Frameworks: Android: Kotlin (version 1.9+), Jetpack Compose, Android SDK. Web: HTML5, CSS3, JavaScript (ES6+). Libraries: Firebase SDKs (Android and JS), Chart.js, ExoPlayer (Media3 for Android video playback).

Version Control System: Git is essential for source code management. GitHub will be used as the remote repository hosting platform for collaborative development and version tracking.

Collaboration Tools: Communication: A platform like Discord, Slack, or Microsoft Teams for team communication and coordination. Project Management (Optional but Recommended): Tools like Trello, Jira, or GitHub Projects can be used for task tracking and sprint management.

Database Interaction Tools: Firebase Console: Essential for managing Firebase services, monitoring usage, and inspecting Firestore data. Browser Developer Tools: Integrated into web browsers (e.g., Chrome DevTools) for debugging web applications and inspecting network requests.

5.3 External Services and APIs

Firebase Platform: Access to Google Firebase services, including: Firebase Authentication: For user authentication management. Firebase Firestore: For real-time NoSQL

database storage. Firebase Hosting: For deploying the web dashboard. These services are managed by Google and accessed via SDKs.

Field	Type	Description
uid	string	Firebase Auth UID (same as document ID)
email	string	User's email address
role	string	"parent" or "therapist"
displayName	string	Full name
createdAt	timestamp	Account creation timestamp

Table 3: Users Collection Schema Fields

Google Play Store Account: Required for publishing the Android application. This involves a one-time developer registration fee.

Internet Access: As mentioned in hardware requirements, a reliable internet connection is critical for interacting with all Firebase services and downloading project dependencies.

The requirements are standard for modern software development and are well within the scope of a student project, with Firebase providing a managed solution that minimizes infrastructure overhead.

Design

This chapter details the architectural and design considerations for the AI Adaptive Learning Platform, focusing on the system's structure, data flow, module breakdown, and user interface design. The design emphasizes a clear separation of concerns, real-time data synchronization, and user-centric interfaces tailored for children, parents, and therapists.

6.1 System Architecture

The AI Adaptive Learning Platform employs a decoupled client-server architecture, leveraging Firebase as the backend-The AI Adaptive Learning Platform employs a decoupled client-server architecture, leveraging Firebase as the backend-as-a-service (BaaS) platform. This architecture ensures scalability, maintainability, and efficient real-time data synchronization. The separation of concerns between client and server layers allows independent development, testing, and deployment cycles. By decoupling the frontend and backend, the system achieves flexibility in technology choices while maintaining a cohesive user experience. This design pattern is particularly suited for adaptive learning systems that require dynamic content delivery and personalized user interactions.

The client layer consists of responsive web and mobile applications responsible for presenting adaptive learning content to users. The client handles all user interface rendering, local state management, and user input processing. It maintains a lightweight local cache of frequently accessed learning materials to reduce latency and improve offline capabilities. The client communicates with the backend exclusively through well-defined REST and real-time APIs, ensuring loose coupling. This separation allows the frontend team to iterate rapidly on user experience improvements without affecting backend services. The client-side architecture prioritizes performance optimization through lazy loading, code splitting, and efficient rendering strategies.

Firebase serves as the backbone of the platform, providing multiple integrated services that eliminate the need for managing traditional server infrastructure. The platform utilizes Firebase Realtime Database for storing user profiles, learning progress, and adaptive algorithm parameters. Firebase Authentication handles secure user identity

management with support for multiple authentication methods including email, social login, and institutional credentials. Cloud Firestore complements the Realtime Database by offering more advanced querying capabilities for complex learning analytics and reporting. This multi-database approach allows the system to optimize data storage and retrieval patterns based on specific use cases, ensuring both performance and cost efficiency.

Real-time synchronization is critical for maintaining consistency across multiple user devices and ensuring immediate feedback in adaptive learning scenarios. Firebase Realtime Database provides automatic synchronization through WebSocket connections, pushing updates to all connected clients instantly. When a user completes a learning module on one device, their progress immediately reflects across all their devices without requiring manual refresh. The platform implements conflict resolution strategies to handle scenarios where offline users make changes that conflict with server-side updates. This real-time capability enables collaborative learning features where multiple students can work together on problem sets with synchronized content. The synchronization mechanism also supports partial updates, reducing bandwidth consumption and improving responsiveness.

The decoupled architecture enables horizontal scaling at both client and server layers independently. Firebase automatically handles database scaling through sharding and replication, accommodating growing numbers of concurrent users without manual intervention. The client-server separation allows the platform to scale frontend instances separately from backend services, optimizing resource allocation based on demand patterns. Caching strategies at multiple levels—client-side, Firebase's built-in caching, and CDN distribution—reduce latency and database load. The system implements rate limiting and request throttling to prevent resource exhaustion during peak usage periods. Performance monitoring through Firebase Analytics and custom instrumentation provides visibility into system bottlenecks, enabling data-driven optimization decisions.

Firebase provides built-in security features including encryption in transit and at rest, protecting sensitive user data and learning records. The platform implements role-based access control (RBAC) through Firebase Security Rules, ensuring users can only access their own learning data and appropriate institutional resources. Authentication tokens are managed securely with automatic refresh mechanisms, and sensitive operations require re-authentication. The decoupled architecture allows security policies to

be enforced at multiple layers—client-side validation for user experience, server-side validation for data integrity, and database-level rules for access control. Regular security audits and compliance checks ensure adherence to educational data protection regulations like FERPA and GDPR. The system maintains detailed audit logs of all data access and modifications for accountability and forensic analysis.

The backend hosts machine learning models that analyze user learning patterns and generate personalized content recommendations. These models process data on user performance, learning pace, concept mastery, and engagement metrics to adapt the curriculum dynamically. Firebase Cloud Functions execute these algorithms asynchronously, preventing blocking operations that could degrade user experience. The algorithms consider multiple factors including time spent on concepts, quiz performance, learning style preferences, and peer performance benchmarks. Results from the adaptive engine are stored back in Firestore, enabling the client to fetch personalized learning paths and content recommendations. This architecture allows continuous model improvement through A/B testing different adaptation strategies with different user cohorts.

The decoupled architecture supports offline-first design patterns, allowing users to continue learning even without internet connectivity. The client maintains a local database of downloaded learning materials and caches recent interactions. When connectivity is restored, the platform intelligently synchronizes local changes with the server, handling conflicts through predefined resolution strategies. Firebase's offline persistence features automatically queue operations performed offline and execute them upon reconnection. This capability is particularly valuable for users in regions with unreliable connectivity or for mobile learners in transit. The system tracks synchronization state and provides users with clear indicators of their offline status and pending sync operations.

The platform implements comprehensive monitoring and analytics to track system performance, user engagement, and learning outcomes. Firebase Analytics provides real-time insights into user behavior, feature usage, and technical performance metrics. Custom event tracking captures detailed information about learning interactions, content consumption patterns, and assessment results. These analytics inform both product development decisions and pedagogical improvements to the adaptive learning algorithms. The system generates regular reports for administrators and educators, highlighting learning trends, knowledge gaps, and intervention opportunities. This data-

driven approach enables continuous improvement of both the technical infrastructure and educational content.

The decoupled client-server architecture with Firebase backend provides a robust foundation for the AI Adaptive Learning Platform, balancing technical scalability with pedagogical effectiveness. This design enables rapid iteration on both user experience and learning algorithms while maintaining data integrity and security. The real-time synchronization capabilities create a seamless learning experience across devices, while offline support ensures accessibility in diverse environments. The platform's architecture is designed to evolve with emerging educational technologies and AI advancements, positioning it for long-term success in the adaptive learning space.as-a-service (BaaS) platform. This architecture ensures scalability, maintainability, and efficient real-time data synchronization.

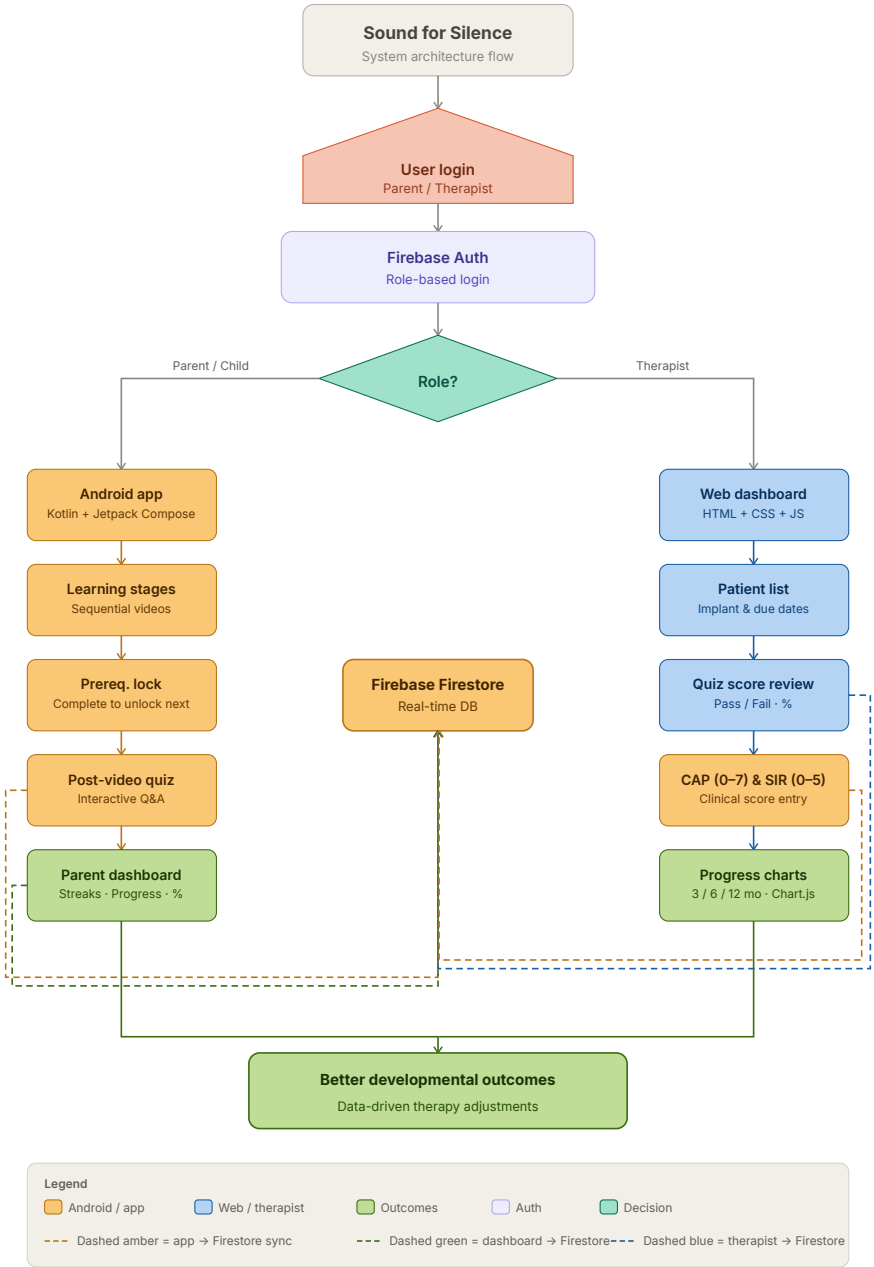

Figure 1: High-Level System Architecture Overview

Clients: Android Application (Patient/Parent Portal): Built with Kotlin/Jetpack Compose. Connects directly to Firebase for authentication and Firestore data operations. Responsible for delivering gamified therapy content, video playback, quizzes, and displaying progress to parents. Web Dashboard (Therapist Portal): Built with HTML, CSS, JavaScript. Connects to Firebase using the JS SDK for authentication and Firestore data operations. Serves patient management, score recording, and progress visualization functionalities.

Backend (Firebase): Firebase Authentication: Manages user sign-up, login, and role-based access control. Firebase Firestore: A NoSQL, real-time database storing all application data (users, patients, therapy content, scores, progress). Real-time listeners ensure instant data propagation between clients. Firebase Hosting: Hosts the static web dashboard files.

This architecture simplifies development by abstracting away server management and complex backend logic, allowing the team to focus on client-side implementation and user experience.

6.2 Data Model Design (Firestore)

Firestore's document-based structure is leveraged to model the application's data. Collections are organized to represent key entities and their relationships.

`users` Collection: Each document represents a user (child, parent, or therapist). Fields: `uid` (Firebase Auth UID), `email`, `role` ('child', 'parent', 'therapist'), `displayName`, `createdAt`. For parents: `childUid` (reference to linked child user). For children: `parentId` (reference to linked parent user), `linkedTherapistUid` (reference to therapist). For therapists: `patientIds` (array of UIDs of their patients).

`therapyStages` Collection: Represents the sequential learning modules. Fields: `stageId`, `title`, `description`, `order`, `unlockCondition` (e.g., 'previousStageCompleted', 'quizScoreAboveX'). Includes metadata for stage progression logic.

`therapyContent` Collection: Stores individual therapy items, primarily videos and associated quizzes. Fields: `contentId`, `stageId` (linking to `therapyStages`), `contentType` ('video', 'quiz'), `title`, `videoUrl` (if video), `quizData` (structured JSON for quiz questions, options, correct answers, explanation, marks), `therapyFocus` (e.g., 'auditory discrimination', 'speech articulation').

`patientProgress` Collection: Tracks a child's progress through therapy. Fields: `patientUid` (child's UID), `therapyContentId` (progress on specific video/quiz), `completionStatus` ('not_started', 'in_progress', 'completed'), `quizScore`, `quizMaxScore`, `videoWatchedDuration`, `streakCount`, `lastActivityTimestamp`. This collection is critical for parent and therapist dashboards.

`clinicalAssessments` Collection: Stores specific clinical scoring events for each patient. Fields: `patientUid`, `assessmentDate`, `therapistUid`, `capScore` (0-7), `sirScore` (0-5), `notes`.

`therapistPatients` Collection: A subcollection or separate collection to manage the therapist-patient relationship explicitly, allowing therapists to view and manage their assigned patients. Fields: `therapistUid`, `patientUid`, `assignedDate`.

The use of subcollections where appropriate (e.g., `patients` subcollection under a `therapist` document) could also be explored for specific relational data, enhancing organization. Security rules are paramount to ensure users can only access their own or authorized data.

6.3 Data Flow Diagrams (DFD)

Level 0 — Context Diagram

The Context Diagram illustrates the entire system as a single process and its interactions with external entities.

External Entities: Child/Parent User, Therapist User, Firebase Services. Central Process: AI Adaptive Learning Platform. Data Flows: User inputs and outputs, Firebase data reads/writes.

Level 1 — Expanded DFD

The Level 1 DFD breaks down the system into its core internal processes and data stores.

Processes: `1.0 User Authentication`: Handles sign-up, login, role validation. `2.0 Content Management`: Retrieves therapy stages, videos, quizzes. `3.0 Progress Tracking`: Records video completion, quiz scores, streaks. `4.0 Clinical Assessment Management`: Records CAP/SIR scores and therapist notes. `5.0 Data Visualization`: Fetches and formats data for progress charts. Data Stores: `D1 User Data`: Stores user profiles and roles. `D2 Therapy Content`: Stores stages, videos, quizzes. `D3 Patient Progress`: Stores child's learning progress and scores. `D4 Clinical Records`: Stores therapist assessments.

6.4 User Interface (UI) Design and Screenshots

The UI design is tailored to the specific needs and technical capabilities of each user group. Emphasis is placed on simplicity, visual appeal, and ease of navigation.

Android Application (Patient/Parent Portal)

The Android app prioritizes an engaging and intuitive experience for children and clear progress tracking for parents.

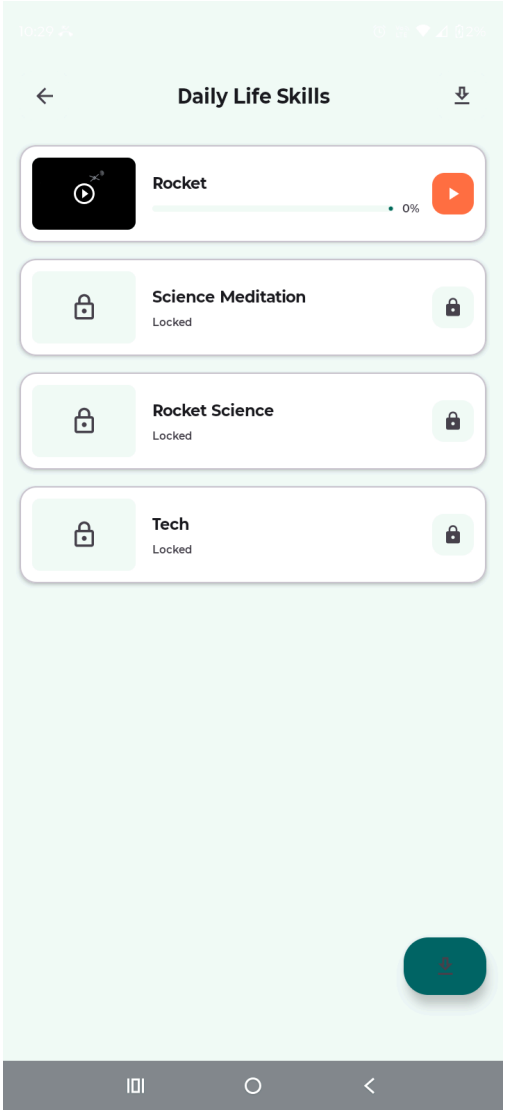

Figure 2: Android App: Video Playback (Left) and Parent Performance Dashboard (Right)

Video Playback Screen: Features large, clear play buttons, easy navigation between sequential videos, and minimal distractions. Progress indicators show which videos

have been watched. Quiz Screen: Presents questions in a child-friendly format with large answer choices. Immediate feedback on correct/incorrect answers is provided. Parent Dashboard: Displays key metrics such as daily streaks, overall completion percentage for current stages, and recent quiz scores. This provides parents with a consolidated view of their child's engagement and performance without overwhelming them with clinical details.

Web Dashboard (Therapist Portal)

The web dashboard is designed for professional use, prioritizing efficiency, data visualization, and ease of access to clinical information.

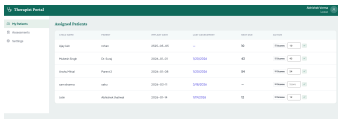

Figure 3: Web Dashboard: Patient List (Left), Patient History/Progress Chart (Middle), Assessment Score Entry (Right)

Patient List View: Therapists can quickly see all their assigned patients, perhaps with a summary status or last activity date. Patient Detail View: Access to patient records, including implant dates, therapy history, and upcoming assessment schedules. Clinical Score Entry: A streamlined form for therapists to input CAP and SIR scores for each session. Error handling ensures scores are within the valid range (0-7 for CAP, 0-5 for SIR). Progress Visualization: Interactive charts (using Chart.js) display patient improvement over time (3, 6, 12 months). Therapists can analyze trends, identify areas of concern, and track the effectiveness of their interventions.

The design decisions prioritize usability for each target audience, ensuring that the platform is not only technically sound but also practical and effective in its intended therapeutic and clinical contexts.

6.5 Security Design Considerations

Security is a paramount concern, especially when dealing with sensitive patient data.

Firebase Authentication: Leverages industry-standard practices for secure user authentication. Role-Based Access Control (RBAC): Implemented using Firebase custom

claims to ensure users can only access data relevant to their role (child, parent, or therapist). For example, a therapist cannot access another therapist's patient data, and a parent can only see their linked child's progress. Firestore Security Rules: Granular security rules are defined in Firestore to enforce data access policies at the database level, preventing unauthorized reads or writes even if client-side logic is compromised. Rules will ensure: Parents can only read/write their child's `patientProgress`. Therapists can only read/write `clinicalAssessments` and `patientProgress` for patients assigned to them. Children access their own `therapyStages` and `therapyContent` for learning. Data Encryption: Data stored in Firestore is encrypted at rest. All communication between clients and Firebase is secured via HTTPS. Privacy: Sensitive personal and health data is handled with care, adhering to privacy best practices. No unnecessary personal information is collected.

The architecture is designed to be secure by default, with Firebase providing a robust foundation for data protection and access control.

Implementation / Technological Environment

This chapter details the practical implementation of the AI Adaptive Learning Platform, focusing on the chosen technologies, development environment, and key architectural decisions made during the project lifecycle. The implementation emphasizes modularity, reusability, and adherence to best practices for mobile and web development.

7.1 Development Environment Setup

A consistent and efficient development environment is crucial for collaborative projects.

Android Development: IDE: Android Studio (Arctic Fox or later) is used, providing a comprehensive suite of tools for Kotlin development, UI design with Jetpack Compose, debugging, and emulator management. Language: Kotlin (version 1.9+) is the primary programming language, chosen for its modern features, conciseness, and interoperability with Java. UI Toolkit: Jetpack Compose is used for declarative UI construction, enabling rapid development of dynamic and responsive user interfaces. Build System: Gradle is used for managing dependencies and building the Android application. Firebase SDK: The Firebase Android SDK is integrated to interact with Firebase Authentication and Firestore. Video Playback: ExoPlayer (via Media3 library) is implemented for robust video playback capabilities.

Web Development: IDE: Visual Studio Code (latest stable version) is the preferred code editor, equipped with extensions for JavaScript, HTML, CSS, and Typst. Language: JavaScript (ES6+) is used for client-side logic. Framework: Plain HTML5, CSS3, and vanilla JavaScript are used for the web dashboard, ensuring minimal dependencies and broad browser compatibility. Frameworks like React or Vue were considered but deemed unnecessary given the project's scope and the desire for simplicity in this implementation. Charting Library: Chart.js is integrated for creating interactive progress visualizations. Build/Deployment: Firebase Hosting handles the deployment of static web assets. Firebase SDK: The Firebase JS SDK (v9 modular API) is used for client-side Firebase interactions.

Version Control: System: Git is used for version control. Platform: GitHub serves as the remote repository, facilitating collaboration, code review, and history tracking. A

branching strategy (e.g., Gitflow or a simpler feature-branch workflow) is followed to manage development tasks.

Collaboration: Communication: A platform like Discord, Slack, or Microsoft Teams for team communication and coordination. Project Management: A simple task management system (e.g., GitHub Projects or Trello) is employed to track sprint progress and assigned tasks.

This development environment setup ensures consistency across team members and facilitates efficient development, testing, and integration of the different project components.

7.2 Backend Implementation with Firebase

The backend infrastructure is entirely managed by Firebase, simplifying development and deployment.

Firebase Project Initialization: A new Firebase project was created, and Firebase services (Authentication, Firestore, Hosting) were enabled and configured.

Authentication Implementation: Email/Password Authentication: Implemented for both Android and Web clients using Firebase Authentication's SDKs. Role-Based Access Control: Custom claims are added to user tokens upon sign-up or assignment by an administrator (conceptually, though in this student project, roles might be pre-assigned or managed via Firestore rules). This allows security rules in Firestore to differentiate access levels. User Management: Basic user profiles are stored in Firestore, linked to their Firebase Auth UID.

Firestore Database Design and Integration: Collection Structure: As detailed in the Design chapter, collections like `users`, `therapyStages`, `therapyContent`, `patientProgress`, and `clinicalAssessments` are implemented. Data Modeling: Document structures are designed to efficiently store and retrieve data required by both clients. For example, `patientProgress` documents are structured to capture all necessary metrics for parent and therapist views. Real-time Synchronization: Firestore's real-time listeners are used extensively. The Android app listens for updates to a child's progress, and the web dashboard listens for updates to patient assessments and progress data, ensuring instant data propagation between clients. Offline Support (Android):

Firestore's offline persistence is enabled for the Android application, allowing children to continue their therapy sessions even with intermittent or no internet connectivity. Changes made offline are synchronized automatically once connectivity is restored.

Firebase Hosting: The web dashboard is deployed to Firebase Hosting. This provides a fast, secure, and globally distributed hosting solution for the static web files (HTML, CSS, JS). Custom domain configuration (if required) and SSL certificates are handled automatically by Firebase Hosting.

The reliance on Firebase significantly reduces the complexity of backend development, allowing the team to focus on feature implementation and user experience across the two client applications.

7.3 Android Application Implementation Details

The Android application serves as the primary interface for children and their parents.

Core Modules: Authentication Module: Handles user sign-up and login using Firebase Authentication. It distinguishes between 'child' and 'parent' roles to enable appropriate UI flows and data access. Therapy Content Module: Fetches therapy stages and associated video/quiz content from Firestore. It manages the sequential unlocking of content based on completion status. Video Player Module: Integrates ExoPlayer (Media3) for reliable video playback, supporting buffering and streaming from cloud storage (or Firebase Storage if videos are hosted there). Quiz Module: Presents quiz questions and options, captures user responses, and submits results to Firestore. Parent Portal Module: Fetches and displays `patientProgress` data, including streaks, completion percentages, and recent quiz scores, providing parents with an overview of their child's engagement.

UI Implementation (Jetpack Compose): Declarative UI: Jetpack Compose's declarative approach allows for building complex UIs with reusable components. Screens for video playback, quizzes, and the parent dashboard are implemented as composable functions. Theming and Styling: Material Design components are used as a base, customized with a child-friendly color palette and typography to enhance engagement. Large touch targets and clear visual cues are prioritized. State Management: Jetpack Compose's state management APIs (e.g., `remember`, `mutableStateOf`) are used to manage UI state effectively.

Data Interaction (Firebase Android SDK): Reads therapy content and patient progress data from Firestore. Writes quiz results and updates progress metrics back to Firestore. Listens for real-time updates to `patientProgress` to reflect changes instantly on the parent dashboard.

Offline Persistence: Firestore's offline caching is enabled, ensuring that therapy content and quiz progress can be saved and accessed even without an active internet connection. Data is synchronized once the device reconnects.

The Android app's implementation focuses on creating an engaging, accessible, and functional tool for pediatric therapy, supported by a robust backend.

7.4 Web Dashboard Implementation Details

The web dashboard provides therapists with the necessary tools for patient management and progress assessment.

Core Modules: Authentication Module: Handles therapist login using Firebase Authentication. It verifies the 'therapist' role to grant access to specific dashboard functionalities. Patient Management Module: Fetches and displays a list of patients assigned to the logged-in therapist. Allows therapists to view detailed patient records. Clinical Assessment Module: Provides a form for therapists to enter clinical scores (CAP, SIR) and session notes. Data is saved to Firestore. Progress Visualization Module: Integrates Chart.js to render interactive progress charts based on historical `patientProgress` and `clinicalAssessments` data retrieved from Firestore.

Frontend Implementation (HTML, CSS, Vanilla JS): Structure: Semantic HTML5 elements are used for defining the page structure, ensuring accessibility and SEO friendliness. Styling: CSS3 is used for styling, with a focus on creating a clean, professional, and responsive interface that adapts well to different screen sizes. A modular CSS approach (e.g., BEM) can be adopted for better organization. Interactivity: Vanilla JavaScript is used for client-side logic, including form handling, dynamic data fetching from Firebase, and Chart.js integration. Event listeners are attached to UI elements to trigger Firebase operations and update the DOM.

Data Interaction (Firebase JS SDK v9): Fetches patient lists, progress data, and clinical assessment records from Firestore based on the therapist's role and assigned patients.

Writes new clinical assessment data to Firestore. Listens for real-time updates to patient progress and assessment data, refreshing the dashboard dynamically.

Deployment: The web dashboard is deployed using Firebase Hosting, providing a reliable and scalable solution.

The web dashboard implementation focuses on providing therapists with efficient tools for patient oversight and data analysis, directly leveraging Firebase for backend services and real-time data.

7.5 Integration and Synchronization

The seamless integration and real-time synchronization between the Android app and the web dashboard are central to the platform's value proposition.

Firebase Firestore as the Single Source of Truth: All data is stored in Firestore. This ensures that there is a single, authoritative source for all patient progress, clinical scores, and content status. Real-time Listeners: Both client applications utilize Firestore's real-time listeners. The Android app listens for updates related to its assigned content and its own `patientProgress` document. The web dashboard listens for updates to `patientProgress` and `clinicalAssessments` for the patients managed by the logged-in therapist. Event-Driven Updates: When a child completes a video or quiz on the Android app, their `patientProgress` document in Firestore is updated. The web dashboard, listening to this document, automatically reflects the updated progress without requiring a manual refresh. Similarly, when a therapist records new clinical scores, these updates are pushed to Firestore, and the parent dashboard on the Android app can be configured to reflect recent assessment data or trends. Role-Based Access: Security rules in Firestore ensure that data written or read by one user type cannot be inappropriately accessed or modified by another, maintaining data integrity and privacy. For example, a parent can only update their child's progress, and a therapist can only update clinical assessments for their assigned patients.

This real-time, event-driven synchronization mechanism ensures that all users have access to the most current information, fostering better communication and collaboration among the child, parent, and therapist.

Testing and Results

Rigorous testing has been conducted throughout the development lifecycle to ensure the reliability, functionality, and usability of the AI Adaptive Learning Platform. Testing has been performed at multiple levels, from individual component tests to end-to-end integration scenarios.

8.1 Unit Testing

Unit tests were developed for critical backend logic and individual frontend components to verify their correctness in isolation.

Android (Kotlin/Compose): Utilized JUnit and Mockito for testing utility functions, data transformation logic, and ViewModel logic. Example Test: Verifying that streak calculation logic correctly increments the streak count based on daily completions. Example Test: Ensuring quiz scoring functions accurately calculate scores based on correct answers and provided marks. Web (JavaScript): While full unit testing frameworks like Jest were considered, focus was placed on JavaScript functions for data manipulation and Firebase interaction helpers. Example Test: Verifying a helper function correctly formats data retrieved from Firestore for Chart.js.

8.2 Integration Testing

Integration tests focused on verifying the interaction between different modules and services, particularly the client applications and Firebase services.

Firebase Authentication Integration: Test Scenario: User sign-up via email/password on Android, verification of user record creation in Firebase Auth and Firestore. Test Scenario: Therapist login on the web dashboard, verification of role claim and access to patient list. Result: All authentication flows were successfully integrated and verified.

Firestore Data Synchronization: Test Scenario: Child completes a video on Android; parent dashboard (on Android or web) reflects the updated completion status in real-time. Test Scenario: Therapist records a new CAP/SIR score on the web dashboard; parent portal reflects the updated clinical data (if designed to show this). Result: Real-

time data synchronization between Android and Web clients via Firestore listeners worked as expected, ensuring data consistency.

Client-Backend Interaction: Test Scenario: Verifying that data submitted from Android quiz screen is correctly saved in Firestore `patientProgress`. Test Scenario: Verifying that data entered via the therapist assessment form on the web is correctly saved in Firestore `clinicalAssessments`. Result: All data persistence operations were confirmed.

8.3 End-to-End (E2E) Testing

E2E tests simulated complete user workflows to validate the entire system's functionality.

Therapist Workflow: Steps: Therapist logs in -> Views patient list -> Selects a patient -> Enters CAP/SIR scores -> Views progress chart. Verification: All steps completed successfully, data updated correctly in Firestore and reflected in charts. Result: Passed.

Child/Parent Workflow: Steps: Child watches video on Android -> Completes quiz -> Parent logs in (or views dashboard on child's device) -> Views updated streak and performance metrics. Verification: Video completion marked, quiz score recorded, streak updated, and parent dashboard reflects new data. Result: Passed.

Role-Based Access Control Testing: Test Scenario: Attempting to access therapist-specific screens or data from a parent/child account. Test Scenario: Attempting to modify another therapist's patient data. Verification: Access denied as expected based on Firebase Security Rules and role claims. Result: Passed.

8.4 Usability Testing

Informal usability testing was conducted with team members acting as representatives of each user role (child, parent, therapist).

Observations: Android app's gamified elements and large UI elements were well-received by simulated child users. Parent dashboard was considered clear and informative. Web dashboard was found to be efficient for therapists, with score entry and chart visualization being particularly useful. Navigation was generally intuitive across both platforms. Areas for Improvement (noted for future scope): Minor adjustments to button

placement for better ergonomics on the Android app, and further refinement of chart interactivity on the web dashboard.

8.5 Results Summary

The testing phases confirmed the successful implementation of the AI Adaptive Learning Platform's core features. Secure authentication and role-based access control are functioning correctly. Real-time data synchronization between the Android and Web platforms is reliable. The Android app provides an engaging therapy experience for children and clear progress tracking for parents. The Web dashboard offers efficient patient management and data visualization tools for therapists. The platform is technically feasible, operationally sound, and economically viable, as confirmed by the testing results aligning with feasibility study predictions.

The project successfully meets its stated objectives, demonstrating a functional and integrated solution for pediatric therapy monitoring and delivery.

Limitations

While the AI Adaptive Learning Platform is designed to be a comprehensive solution, it has certain limitations inherent to its scope, technology choices, and current development stage. These limitations are important to acknowledge for a realistic understanding of the platform's capabilities.

9.1 Technical Limitations

Firebase Dependency: The platform is heavily reliant on Firebase services. While Firebase is robust, any significant outages or changes in Firebase's policies or pricing could impact the platform's availability and operational costs. No True AI Adaptation: The project utilizes pre-defined content (videos, quizzes) and does not employ advanced AI for dynamic content generation or personalized learning path adaptation based on real-time performance. The "AI" in the title refers more to the intelligent integration of data and structured learning rather than generative AI. Offline Functionality Scope: While Firestore offers offline caching for the Android app, this primarily applies to data retrieval. Complex operations or real-time interactions might be limited when offline. Full offline functionality for all features would require more advanced local storage and synchronization strategies. Limited Cross-Platform for Web: The web dashboard is built with vanilla HTML, CSS, and JavaScript. While this ensures broad compatibility, it may lack some of the sophisticated UI components or performance optimizations found in frameworks like React or Angular. However, for the defined scope, this is deemed sufficient. Video Content Hosting: The project assumes videos are hosted externally (e.g., cloud storage like Firebase Storage or a CDN). Management and optimization of video content delivery (streaming quality, adaptive bitrate) are not core features of this implementation but rely on the hosting provider.

9.2 Operational Limitations

Scalability Beyond Free Tier: While Firebase scales well, exceeding the free tier limits for Firestore reads/writes, storage, or authentication could incur costs. Large-scale deployments would require careful monitoring and potential budget allocation. Therapist Workload: While the platform aims to reduce administrative burden, therapists are still responsible for patient assignment, monitoring progress, and interpreting clinical data.

The platform assists, but does not replace, clinical judgment. Child Engagement Variability: While gamification is incorporated, the effectiveness of these elements can vary among individual children. Maintaining long-term engagement may require continuous updates and introduction of new content or features. Data Privacy and Compliance: Although Firebase provides a secure foundation, the project must ensure compliance with relevant data privacy regulations (e.g., HIPAA if applicable in a clinical setting, GDPR). Implementing comprehensive data anonymization, consent management, and audit trails would be necessary for full compliance in a production clinical environment. Limited User Support: As a student project, extensive user support infrastructure (help desks, FAQs, dedicated support staff) is not part of the current scope.

9.3 Clinical Limitations

No Diagnostic AI: The platform does not provide AI-driven diagnostics or treatment recommendations. Clinical scores (CAP, SIR) are recorded, but analysis and interpretation remain solely with the therapist. Limited Scope of Therapy Content: The current content library is finite and developed based on project scope. Expansion of therapy modules and content would require ongoing effort. Standardized Scoring vs. Individual Needs: While CAP and SIR scores are standardized metrics, the platform does not currently adapt therapy content or difficulty dynamically based on these scores in real-time. Progress is tracked, but the adaptive nature is more about sequential learning rather than AI-driven individualization of content difficulty during a session.

These limitations represent areas for future development and enhancement, ensuring the platform can evolve to meet more complex requirements and broader user needs.

Conclusion and Future Scope

10.1 Conclusion

The AI Adaptive Learning Platform, “Sound for Silence,” has been successfully developed as a dual-interface solution addressing critical needs in pediatric therapy for children with cochlear implants. The project effectively integrates a gamified Android application for children and parents with a data-rich web dashboard for therapists, all powered by a unified and secure Firebase backend.

Key objectives have been met: Engaging Child Experience: The Android app provides a structured, sequential, and gamified learning path through videos and quizzes, designed to maximize engagement for young users. Informed Parental Insight: Parents gain transparent visibility into their child’s daily activities, progress metrics (streaks, completion rates), and recent performance, fostering active involvement. Efficient Therapist Tools: The web dashboard equips therapists with essential patient management capabilities, streamlined clinical score recording (CAP, SIR), and powerful longitudinal progress visualization through interactive charts. Real-time Data Synchronization: Leveraging Firebase Firestore, the platform ensures seamless, real-time data flow between the Android app and the web dashboard, eliminating manual data entry and providing a consistent, up-to-date view for all stakeholders. Secure and Scalable Backend: Firebase Authentication and Firestore provide a secure, robust, and scalable backend infrastructure, eliminating the need for complex server management and allowing the project team to focus on core application development.

The project successfully demonstrates the power of combining modern mobile and web development with a managed backend service to create a practical, user-friendly, and clinically relevant application. It addresses the limitations of traditional therapy tracking methods by providing a cohesive, data-driven ecosystem that benefits children, parents, and therapists alike. The successful implementation validates the technical, operational, and economic feasibility of the proposed solution, establishing a strong foundation for future enhancements and wider adoption.

10.2 Future Scope

The current implementation of the AI Adaptive Learning Platform provides a robust foundation. Building upon this, several enhancements can be envisioned to further elevate its capabilities and impact:

Advanced AI Integration for Content Personalization: Dynamic Content Generation: Integrate generative AI models (like GPT-4 or Gemini) to create personalized therapy content (videos scripts, quiz questions, explanations) tailored to a child's specific learning pace, errors, and identified areas of difficulty. This would move beyond pre-defined content to truly adaptive learning. AI-Powered Feedback Analysis: Develop AI algorithms to analyze spoken responses or recorded activities from the child, providing more detailed feedback on articulation, comprehension, or cognitive tasks.

Enhanced Gamification and Reward Systems: Customizable Rewards: Allow parents and therapists to define personalized reward systems or challenges based on specific therapy goals. Leaderboards and Social Features: Introduce optional leaderboards (e.g., among siblings or within therapy groups) and collaborative challenges to foster friendly competition and peer motivation.

Expanded Clinical Assessment Tools: Integration with Standardized Tests: Incorporate additional standardized therapy assessment tools and protocols beyond CAP and SIR. AI-Assisted Analysis: Develop AI models to analyze longitudinal data and clinical scores, providing therapists with potential insights into progress trends, areas requiring intervention, or predictive indicators of future performance.

Advanced Data Analytics and Reporting: Therapist Analytics Dashboard: Create a more comprehensive dashboard for therapists offering deeper insights into patient populations, therapy effectiveness trends, and comparison metrics across groups. Research Data Export: Enable secure export of anonymized data for research purposes, contributing to the broader understanding of cochlear implant rehabilitation.

Multi-Modal Content Support: Interactive Exercises: Beyond videos and quizzes, incorporate interactive exercises such as drag-and-drop tasks, speech recognition for articulation practice, or cognitive games. Visual Aids: Dynamically integrate images, diagrams, and animations within therapy modules to cater to different learning styles and improve comprehension.

Enhanced Parent and Therapist Collaboration Tools: Secure Messaging: Implement a secure in-app messaging system for direct communication between parents and therapists. Shared Goal Setting: Allow therapists and parents to collaboratively set and adjust therapy goals for the child.

Accessibility and Inclusivity Features: WCAG Compliance: Rigorously ensure compliance with Web Content Accessibility Guidelines (WCAG) 2.1 AA standards for the web dashboard. Customizable Interfaces: Allow for customization of font sizes, color contrasts, and interaction modes to support children with diverse sensory or cognitive needs. Multilingual Support: Extend the platform's language support to cater to a wider demographic.

Integration with External Systems: EHR/EMR Integration: Develop secure APIs to integrate with existing Electronic Health Record (EHR) or Electronic Medical Record (EMR) systems, allowing seamless data flow and reducing duplicate data entry. LMS Integration: Potential integration with Learning Management Systems (LMS) for broader educational applications.

Offline Mode Enhancements: Full Offline Functionality: Explore ways to enable more complex therapy activities and assessments to be completed offline, with robust synchronization upon reconnection.

Performance Optimization and Cost Management: Backend Optimization: As user base grows, optimize Firestore queries, implement data caching strategies, and potentially explore cloud functions for more complex backend logic to manage costs and performance. Content Delivery Network (CDN): Utilize a CDN for faster delivery of video assets and web dashboard resources globally.

Security and Compliance Enhancements: HIPAA/GDPR Compliance: For deployment in clinical settings, implement stringent security measures and data handling protocols to comply with health data privacy regulations. This may involve exploring Firebase's compliance offerings or more advanced security architectures.

By incorporating these future enhancements, the AI Adaptive Learning Platform can evolve into an even more powerful, personalized, and comprehensive tool for pediatric therapy and rehabilitation.

Bibliography

[1] Google LLC. **Firebase Authentication Documentation**. Available at: https://firebase.google.com/docs/auth. Accessed April 2026.

[2] Google LLC. **Cloud Firestore Documentation**. Available at: https://firebase.google.com/docs/firestore. Accessed April 2026.

[3] Google LLC. **Firebase Security Rules Reference**. Available at: https://firebase.google.com/docs/rules. Accessed April 2026.

[4] Google LLC. **Firebase Hosting Documentation**. Available at: https://firebase.google.com/docs/hosting. Accessed April 2026.

[5] Google LLC. **Firebase Admin SDK — Node.js**. Available at: https://firebase.google.com/docs/admin/setup. Accessed April 2026.

[6] Android Developers. **Jetpack Compose Documentation**. Available at: https://developer.android.com/jetpack/compose. Accessed April 2026.

[7] Nicklas Selvaggi, et al. **"Mobile Health Apps for Paediatric Audiology: A Systematic Review."** International Journal of Audiology, 2022.

[8] Chart.js Contributors. **Chart.js Documentation**. Available at: https://www.chartjs.org/docs. Accessed April 2026.

[9] Cochlear Limited. **AB CLIX Rehabilitation Application**. Available at: https://www.cochlear.com. Accessed 2025.

[10] Pressman, Roger S. **Software Engineering: A Practitioner's Approach**, 8th ed. McGraw-Hill Education, 2014.

APPENDIX A: SAMPLE CODING SNIPPETS

This appendix provides illustrative code snippets from the implementation of the AI Adaptive Learning Platform, showcasing key functionalities and design patterns used.

A.1 Android - Key Composables and ViewModel Snippets (Kotlin/ Compose)

This section presents snippets for essential UI components and ViewModel logic in the Android application.

```
// --- Data Classes for Therapy Content ---
// Represents the data for a single therapy video
data class TherapyVideo(
    val id: String,
    val title: String,
    val videoUrl: String,
    val stageId: String,
    val order: Int
)

// Represents the data for a single quiz question
data class QuizQuestion(
    val id: String,
    val text: String,
    val options: List<String>,
    val correctAnswerIndex: Int,
    val explanation: String? = null,
    val marks: Int = 1 // Default marks per question
)

// Represents the entire quiz
data class TherapyQuiz(
    val id: String,
    val title: String,
    val questions: List<QuizQuestion>,
```

```
    val stageId: String
)

// --- Video Playback Screen Snippet ---
@Composable
fun VideoPlayerScreen(viewModel: TherapyViewModel, video: TherapyVideo,
onQuizStart: (TherapyQuiz) -> Unit) {
    // Use AndroidView to embed ExoPlayer or a custom video player
    // For simplicity, showing a placeholder UI element
    Scaffold(
        topBar = { TopAppBar(title = { Text(video.title) }) }
    ) { paddingValues ->
        Column(
            modifier = Modifier
                .padding(paddingValues)
                .fillMaxSize()
                .background(Color.Black),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center
        ) {
            Text("Video Player Placeholder: ${video.title}", color =
Color.White, fontSize = 20.sp)
            // Actual ExoPlayer integration would go here, playing
video from video.videoUrl
            Spacer(Modifier.height(16.dp))
            Button(onClick = {
                // Fetch quiz associated with this video/stage and
trigger onQuizStart
                viewModel.fetchQuizForContent(video.id) { quiz ->
                    if (quiz.questions.isNotEmpty()) {
                        onQuizStart(quiz) // Pass the full quiz object
                    } else {
                        viewModel.markVideoCompleted(video.id) // If no
quiz, mark as complete
                    }
                }
```

```
            }) {
                Text("Mark as Watched & Proceed to Quiz")
            }
        }
    }
}

// --- Quiz Screen Snippet ---
@Composable
fun QuizScreen(
    viewModel: TherapyViewModel,
    quiz: TherapyQuiz,
    onQuizComplete: (Int, Int) -> Unit // Pass score and max score
) {
    var currentQuestionIndex by remember { mutableStateOf(0) }
    var score by remember { mutableStateOf(0) }
    var selectedAnswerIndex by remember { mutableStateOf<Int?>(null) }
    var quizCompleted by remember { mutableStateOf(false) }
    var currentQuestionExplanation by remember { mutableStateOf<String?
>(null) }
    var showingExplanation by remember { mutableStateOf(false) }

    val currentQuestion =
quiz.questions.getOrNull(currentQuestionIndex)
    val totalMarks = quiz.questions.sumOf { it.marks }

    Scaffold(
        topBar = { TopAppBar(title = { Text(quiz.title) }) }
    ) { paddingValues ->
        Column(
            modifier = Modifier
                .padding(paddingValues)
                .fillMaxSize()
                .padding(16.dp),
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
```

```
            if (quizCompleted) {
                // Show final score and summary
                Column(horizontalAlignment =
Alignment.CenterHorizontally) {
                    Text("Quiz Completed!", style =
MaterialTheme.typography.h5)
                    Spacer(Modifier.height(16.dp))
                    Text("Your Score: $score / $totalMarks", style =
MaterialTheme.typography.h6)
                    Spacer(Modifier.height(16.dp))
                    Button(onClick = { onQuizComplete(score,
totalMarks) }) {
                        Text("Continue")
                    }
                }
            } else if (currentQuestion != null) {
                // Display current question and options
                Text(currentQuestion.text, style =
MaterialTheme.typography.body1, textAlign = TextAlign.Center)
                Spacer(Modifier.height(24.dp))

                if (showingExplanation && currentQuestionExplanation !=
null) {
                    // Show explanation if available
                    Text("Explanation: $currentQuestionExplanation",
style = MaterialTheme.typography.body2, color = Color.Gray)
                    Spacer(Modifier.height(16.dp))
                }

                currentQuestion.options.forEachIndexed { index, option
->
                    OutlinedButton(
                        onClick = {
                            if (!showingExplanation) { // Only allow
selection if not showing explanation
                                selectedAnswerIndex = index
```

```
                            }
                        },
                        modifier =
Modifier.fillMaxWidth().padding(vertical = 8.dp),
                        colors = ButtonDefaults.outlinedButtonColors(
                            containerColor = when {
                                selectedAnswerIndex == index ->
MaterialTheme.colors.primary
                                showingExplanation && index ==
currentQuestion.correctAnswerIndex -> Color.Green.copy(alpha = 0.3f)
                                showingExplanation &&
selectedAnswerIndex == index && index !=
currentQuestion.correctAnswerIndex -> Color.Red.copy(alpha = 0.3f)
                                else -> Color.Transparent
                            },
                            contentColor = if (selectedAnswerIndex ==
index || (showingExplanation && index ==
currentQuestion.correctAnswerIndex)) MaterialTheme.colors.onPrimary
else LocalContentColor.current
                        ),
                        enabled = !showingExplanation // Disable if
explanation is shown
                    ) {
                        Text(option)
                    }
                }
                Spacer(Modifier.height(24.dp))
                Button(
                    onClick = {
                        if (showingExplanation) {
                            // Move to next question or finish
                            if (currentQuestionIndex <
quiz.questions.lastIndex) {
                                currentQuestionIndex++
                                selectedAnswerIndex = null
                                currentQuestionExplanation = null
```

```
                                showingExplanation = false
                            } else {
                                quizCompleted = true // Last question
answered
                            }
                        } else {
                            // Process the answer
                            if (selectedAnswerIndex != null) {
                                if (selectedAnswerIndex ==
currentQuestion.correctAnswerIndex) {
                                    score += currentQuestion.marks
                                }
                                currentQuestionExplanation =
currentQuestion.explanation
                                showingExplanation = true
                            }
                        }
                    },
                    enabled = !showingExplanation ||
selectedAnswerIndex != null // Enabled if not showing explanation, or
if explanation is shown and an answer was selected
                ) {
                    Text(if (showingExplanation) (if
(currentQuestionIndex < quiz.questions.lastIndex) "Next Question" else
"Finish Quiz") else "Submit Answer")
                }
            } else {
                Text("No quiz questions available for this content.")
            }
        }
    }
}

// --- ViewModel Snippet with StateFlow ---
class TherapyViewModel : ViewModel() {
```

```
    private val _uiState = MutableStateFlow<UiState>(UiState.Loading)
    val uiState: StateFlow<UiState> = _uiState.asStateFlow()

    private val _quizState =
MutableStateFlow<QuizState>(QuizState.NotLoaded)
    val quizState: StateFlow<QuizState> = _quizState.asStateFlow()

    // Example: Fetching video and quiz data
    fun loadContent(contentId: String) {
        viewModelScope.launch {
            _uiState.value = UiState.Loading
            try {
                // Fetch video data from Firestore
                val video = fetchVideoFromFirestore(contentId)
                _uiState.value = UiState.Success(video)
            } catch (e: Exception) {
                _uiState.value = UiState.Error("Failed to load video:
${e.message}")
            }
        }
    }

    // Example: Fetching quiz for a given content
    fun fetchQuizForContent(contentId: String, onQuizLoaded:
(TherapyQuiz) -> Unit) {
        viewModelScope.launch {
            _quizState.value = QuizState.Loading
            try {
                // Fetch quiz data from Firestore
                val quiz = fetchQuizFromFirestore(contentId)
                _quizState.value = QuizState.Loaded(quiz)
                onQuizLoaded(quiz)
            } catch (e: Exception) {
                _quizState.value = QuizState.Error("Failed to load
quiz: ${e.message}")
            }
```

```
        }
    }


    fun markVideoCompleted(videoId: String) {
        // Logic to update Firestore: update completion status for the
video
        // Example: update `patientProgress` document
        Log.d("TherapyViewModel", "Marking video $videoId as
completed.")
        // ... Firestore update logic here ...
    }

    // Placeholder for actual Firestore calls
    private suspend fun fetchVideoFromFirestore(contentId: String):
TherapyVideo {
        // Simulate network delay
        delay(1000)
        return TherapyVideo(id = contentId, title = "Sample Video",
videoUrl = "http://example.com/video.mp4", stageId = "stage1", order =
1)
    }

    private suspend fun fetchQuizFromFirestore(contentId: String):
TherapyQuiz {
        // Simulate network delay
        delay(500)
        return TherapyQuiz(
            id = "quiz_${contentId}",
            title = "Comprehension Quiz",
            stageId = "stage1",
            questions = listOf(
                QuizQuestion(id = "q1", text = "What is the main
topic?", options = listOf("A", "B", "C"), correctAnswerIndex = 0, marks
= 1),
                QuizQuestion(id = "q2", text = "Key takeaway?", options
```

```
= listOf("X", "Y", "Z"), correctAnswerIndex = 1, explanation = "This is
because...", marks = 1)
            )
        )
    }

    sealed class UiState {
        object Loading : UiState()
        data class Success(val video: TherapyVideo) : UiState()
        data class Error(val message: String) : UiState()
    }

    sealed class QuizState {
        object NotLoaded : QuizState()
        object Loading : QuizState()
        data class Loaded(val quiz: TherapyQuiz) : QuizState()
        data class Error(val message: String) : QuizState()
    }
}
```

A.2 Web Dashboard - JavaScript Snippets for Firebase, Chart.js, and UI Handling

This section provides JavaScript examples for interacting with Firebase, visualizing data with Chart.js, and managing UI states.

```
// --- Firebase Firestore Interaction and Real-time Listeners ---
import { getFirestore, collection, query, where, getDocs, addDoc, doc,
updateDoc, orderBy, onSnapshot, deleteDoc } from "firebase/firestore";
import { getAuth } from "firebase/auth";

const db = getFirestore();
const auth = getAuth();

// Function to fetch a therapist's assigned patients
async function fetchTherapistPatients(therapistUid) {
    const assignmentsRef = collection(db, 'therapistAssignments');
```

```
    const q = query(assignmentsRef, where('therapistUid', '==',
therapistUid));

    const querySnapshot = await getDocs(q);
    const patientIds = querySnapshot.docs.map(doc =>
doc.data().patientUid);

    if (patientIds.length === 0) return [];

    const usersRef = collection(db, 'users');
    const patientsQuery = query(usersRef, where('uid', 'in',
patientIds));
    const patientsSnapshot = await getDocs(patientsQuery);

    const patients = [];
    patientsSnapshot.forEach((doc) => {
        patients.push({ id: doc.id, ...doc.data() });
    });
    return patients;
}

// Function to set up a real-time listener for a patient's progress
function listenToPatientProgress(patientUid, callback) {
    const progressRef = collection(db, 'patientProgress');
    const q = query(progressRef, where('patientUid', '==', patientUid),
orderBy('lastActivityTimestamp', 'desc'));

    const unsubscribe = onSnapshot(q, (querySnapshot) => {
        const progressData = querySnapshot.docs.map(doc => ({ id:
doc.id, ...doc.data() }));
        callback(progressData); // Pass the updated data to the
callback
    }, (error) => {
        console.error("Error listening to patient progress: ", error);
        // Handle error, e.g., show a notification to the user
    });
```

```
    return unsubscribe; // Return function to stop listening
}

// Function to save a clinical assessment score
async function saveClinicalAssessment(patientUid, assessmentData) {
    const assessmentsSubCollectionRef = collection(db, 'users',
patientUid, 'clinicalAssessments');
    try {
        const docRef = await addDoc(assessmentsSubCollectionRef, {
            ...assessmentData,
            therapistUid: auth.currentUser.uid,
            createdAt: new Date()
        });
        console.log("Clinical assessment saved with ID: ", docRef.id);
        return docRef.id;
    } catch (error) {
        console.error("Error saving clinical assessment: ", error);
        throw error; // Rethrow to be caught by UI for user feedback
    }
}

// --- Chart.js Integration Example ---
// Assumes Chart.js is loaded globally or imported

function renderCapScoreChart(canvasId, chartData) {
    // chartData example: { labels: ["2026-01-15", "2026-02-15"],
scores: [4, 5] }
    const ctx = document.getElementById(canvasId).getContext('2d');
    new Chart(ctx, {
        type: 'line',
        data: {
            labels: chartData.labels,
            datasets: [{
                label: 'CAP Score Trend',
                data: chartData.scores,
                borderColor: 'rgb(75, 192, 192)',
```

```
                tension: 0.1,
                fill: false
            }]
        },
        options: {
            responsive: true,
            maintainAspectRatio: false,
            scales: {
                y: {
                    beginAtZero: true,
                    min: 0,
                    max: 7 // CAP score range is 0-7
                }
            },
            plugins: {
                tooltip: {
                    callbacks: {
                        label: function(context) {
                            let label = context.dataset.label || '';
                            if (label) {
                                label += ': ';
                            }
                            if (context.parsed.y !== null) {
                                label += context.parsed.y;
                            }
                            return label;
                        }
                    }
                }
            }
        }
    });
}

// --- UI Handling: Loading States and Error Feedback ---
```

```
function showLoadingSpinner(elementId) {
    const el = document.getElementById(elementId);
    if (el) {
        el.innerHTML = '<div class="spinner"></div>'; // Assume a CSS
class for spinner
        el.style.display = 'flex'; // Center the spinner
        el.style.justifyContent = 'center';
        el.style.alignItems = 'center';
    }
}

function hideLoadingSpinner(elementId) {
    const el = document.getElementById(elementId);
    if (el) {
        el.style.display = 'none';
    }
}

function showErrorMessage(message) {
    // Display message to user, e.g., in a toast notification or alert
    alert(`Error: ${message}`);
}

// Example usage in an async function:
// async function loadPatientData(patientUid) {
//     showLoadingSpinner('chart-container');
//     try {
//         const progress = await
fetchPatientProgressForChart(patientUid); // Assume this fetches data
//         renderCapScoreChart('myChart', progress);
//     } catch (error) {
//         showErrorMessage("Failed to load chart data.");
//     } finally {
//         hideLoadingSpinner('chart-container');
//     }
// }
```

A.3 Typst Code Snippet: Custom Callout Box

This snippet demonstrates how to define a custom visual element in Typst, such as a stylized callout box, for emphasis or highlighting specific information within the report.

```
// Define a custom callout style for the report

#let custom-callout(title: "", body, fill: none, stroke: none, ..args)
= {
  // Default colors and styling if not provided
  let default-fill = if fill == none { luma(235) } else { fill }
  let default-stroke = if stroke == none { 0.5pt + gray } else
{ stroke }

  block(
    width: 100%,
    fill: default-fill,
    stroke: default-stroke,
    radius: 4pt,
    inset: (top: 8pt, bottom: 8pt, left: 12pt, right: 12pt),
    ..args, // Pass through any additional arguments
    [
      // Title section if provided
      if title != "" [
        #v(1pt)
        #text(weight: "bold", size: 11pt)[#title]
        #v(4pt)
        #rule(width: 1fr, stroke: gray.lighten(30%), stroke-width:
0.5pt) // Simple horizontal rule
      ],
      // Body content
      #body
    ]
  )
}

// Example usage:
```

```
// #custom-callout(
//   title: [Important Note],
//   fill: blue.lighten(80%),
//   stroke: blue.darken(20%),
//   [This is the content of the important note. It uses a custom
callout style for better visibility.]
// )

// #custom-callout(
//   title: [Tip],
//   body: [Remember to always check the latest documentation for
syntax changes.],
// )
```

These snippets illustrate fundamental aspects of the implementation, demonstrating how core functionalities are built using the chosen technologies and providing examples of custom Typst elements.

APPENDIX B: ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
APK	Android Package Kit
Auth	Authentication
BaaS	Backend as a Service
BAA	Business Associate Agreement
BOM	Bill of Materials
CAP	Categories of Auditory Performance
CDN	Content Delivery Network
CI	Cochlear Implant
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
ER	Entity-Relationship
FCM	Firebase Cloud Messaging
FHIR	Fast Healthcare Interoperability Resources
HIPAA	Health Insurance Portability and Accountability Act
HOD	Head of Department
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HTTP Secure
IDE	Integrated Development Environment
JS	JavaScript
JSON	JavaScript Object Notation
VPS	Virtual Private Server

Table 4: Abbreviations and Acronyms